JAVASCRIPT

Documentación Técnica

Generado: 16/10/2026 02:40 UTC

AutoDocs AI

Tabla de Contenido

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 7

2.5 Diagrama de Casos de Uso . . . 7

2.6 Diagrama de Clases . . . 7

2.7 Patrones Detectados . . . 7

2.8 Insights de Arquitectura (IA) . . . 7

3. Documentación Técnica de Código . . . 7

3.1 Guía de Configuración del Entorno de Desarrollo . . . 7

3.2 Documentación de API . . . 8

3.3 Estado de Comentarios y Docstrings . . . 9

3.4 Planes de Prueba (Testing) . . . 10

4. Clases . . . 10

5. Documentación de Funciones . . . 10

Módulo: `backend/server.js` . . . 10

Módulo: `backend/words.js` . . . 12

Módulo: `backend/test.js` . . . 13

Módulo: `mobile/App.js` . . . 13

Módulo: `mobile/components/HangmanDrawing.js` . . . 13

Módulo: `mobile/components/Keyboard.js` . . . 13

Módulo: `mobile/app/GameScreen.js` . . . 14

Módulo: `mobile/app/HomeScreen.js` . . . 17

Módulo: `mobile/app/WaitingScreen.js` . . . 17

Módulo: `mobile/data/words.js` . . . 18

Módulo: `mobile/hooks/useSocket.js` . . . 18

6. Modelos de Datos . . . 19

Esquema General Inferido . . . 19

7. Guía de Despliegue y Mantenimiento . . . 19

7.1 Requisitos del Sistema . . . 19

7.2 Instalación Local . . . 19

7.3 Despliegue con Docker . . . 19

7.4 Variables de Entorno Recomendadas . . . 20

7.5 Checklist de Despliegue . . . 20

7.6 Registro de Cambios (Changelog) . . . 20

8. Reporte de Calidad del Código . . . 21

8.1 Análisis de Métricas . . . 21

8.2 Funciones con Alta Complejidad Ciclomática . . . 21

8.3 Recomendaciones . . . 21

8.4 Errores de Análisis . . . 21

9. Análisis de Datos del Proyecto . . . 21

9.1 Tipos y Fuentes de Datos Detectados 22

9.2 Esquema de Data Warehouse Inferido 22

9.3 Reporte de Calidad de Datos 22

9.4 Minería de Datos Aplicada al Código 22

9.5 Pipeline ETL Utilizado 24

JavaScript

Documentación Técnica · AutoDocs AI · 16/10/2026

21	42	0	3
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 71/100

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-10-16 02:39 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **JavaScript** y cuenta con 21 archivos fuente, 42 funciones, 0 clases y 3 endpoints de API documentados. El score de calidad del código es **71/100**.

Stack Tecnológico

Lenguaje	Cantidad
SQL	1 archivos
JSON	6 archivos
JavaScript	14 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	21
Funciones	42
Clases	0
Endpoints API	3
Score de calidad	71/100

Funciones documentadas	0 / 42
Clases documentadas	0 / 0

---

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ backend/
■■■ mobile/
```

2.2 Distribución por Capas

• **`(raíz)/`** — 1 archivos [.gitignore(1)]

• **`backend/`** — 9 archivos [.example(1), .sql(1), .json(2), .sin ext(1), .js(3), .env(1)]

• **`mobile/`** — 15 archivos [.js(11), .json(4)]

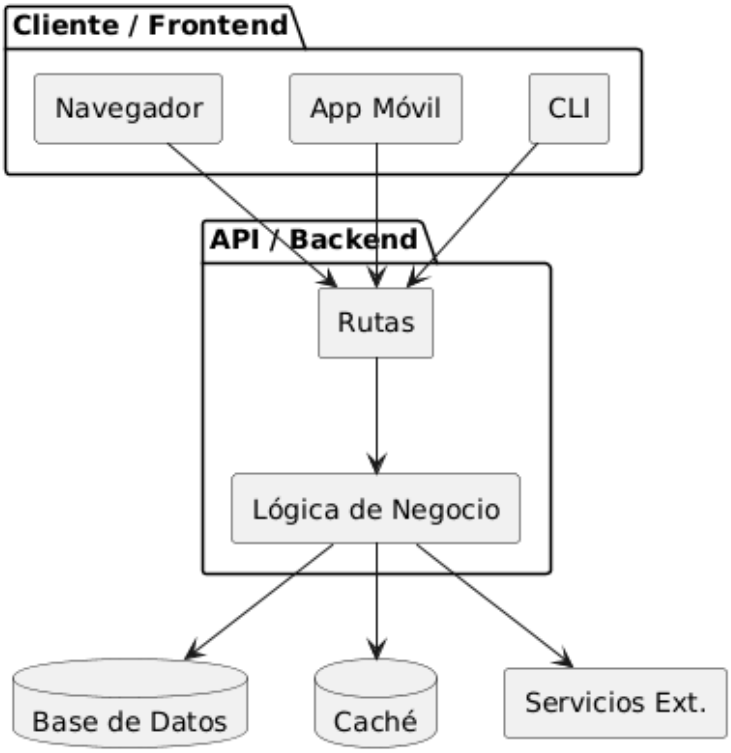
2.3 Distribución de Lenguajes

• JavaScript: 14 archivos (67%)

• JSON: 6 archivos (29%)

• SQL: 1 archivos (5%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases

2.7 Patrones Detectados

• **API REST**: 3 endpoints detectados (Express.js)

• **Testing**: 1 funciones de prueba detectadas

• **Programación Asíncrona**: 12 funciones async/await

2.8 Insights de Arquitectura (IA)

No se pudieron generar insights de arquitectura

---

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
npm install

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **3**

Módulo: `backend/server.js`

[GET] `GET /health`

• **Framework**: Express.js

• **Archivo**: `backend/server.js`

Endpoint GET /health

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[POST] `POST /rooms`

• **Framework**: Express.js

• **Archivo**: `backend/server.js`

Endpoint POST /rooms

Parámetros esperados:

• `body` (JSON, requerido): Cuerpo de la solicitud

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[POST] `POST /rooms/:code/join`

• **Framework**: Express.js

• **Archivo**: `backend/server.js`

Endpoint POST /rooms/:code/join

Parámetros esperados:

• `body` (JSON, requerido): Cuerpo de la solicitud

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 42

• **[OK] Documentadas:** 0 (0%)

• **[X] Sin documentar:** 42 (100%)

Visualización de documentación:

```
[                        ]
0% documentado
```

Funciones que requieren documentación:

• `generateRoomCode()` en `backend/server.js`

• `getRoomByCode(code)` en `backend/server.js`

• `getUsedWords(roomId)` en `backend/server.js`

• `maskWord(word, guessedLetters)` en `backend/server.js`

• `checkWin(word, guessedLetters)` en `backend/server.js`

• ... y 37 más

3.4 Planes de Prueba (Testing)

Se detectaron **1 funciones de prueba**:

Función de Test	Archivo
`testConnection`	`backend/test.js`

---

4. Clases

No se detectaron clases en este proyecto.

---

5. Documentación de Funciones

Total: **42 funciones**

Módulo: `backend/server.js`

`generateRoomCode`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Función generateRoomCode

`async getRoomByCode`

• Parámetros: `code`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Función getRoomByCode

`async getUsedWords`

• Parámetros: `roomId`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Función getUsedWords

`maskWord`

• Parámetros: `word, guessedLetters`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Función maskWord

`checkWin`

• Parámetros: `word, guessedLetters`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Función checkWin

`serializeRoom`

• Parámetros: `room`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async startRound`

• Parámetros: **`roomCode`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `backend/words.js`

`isValidWord`

• Parámetros: **`word`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`normalizeWord`

• Parámetros: **`word`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async fetchFromPalabrasDev`

• Parámetros: **`_`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`async fetchBatchFromPalabrasDev`

• Parámetros: **`count = 5`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`async getRandomWord`

• Parámetros: `usedWords = []`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `backend/test.js`

`async testConnection`

• Parámetros: `—`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/App.js`

`App`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/components/HangmanDrawing.js`

`HangmanDrawing`

• Parámetros: `{ wrongAttempts, size = 200 }`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/components/Keyboard.js`

`Keyboard`

• Parámetros: **`{ guessedLetters, onGuess, disabled, currentWord }`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`getLetterStatus`

• Parámetros: **`letter`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`renderKey`

• Parámetros: **`letter`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/app/GameScreen.js`

`useSounds`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`buildInitialSPState`

• Parámetros: **`{ word, playerName, difficulty, hintsCount, playerWins }`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`GameScreen`

• Parámetros: **`{ route, navigation }`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`clearTimerInterval`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`resetTimer`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`startTimer`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`StatCell`

• Parámetros: `{ icon, color, label, value }`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`shakeScreen`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`bounceWin`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`flashLoss`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`pulseLetter`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`handleHint`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`processGuess`

• Parámetros: `letter, isHint = false`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`handleGuess`

• Parámetros: `letter`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`handleExit`

• Parámetros: `confirmed = false`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async handleNewGame`

• Parámetros: `—`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/app/HomeScreen.js`

`HomeScreen`

• Parámetros: `{ route, navigation }`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async createRoom`

• Parámetros: `—`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`async joinRoom`

• Parámetros: `—`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/app/WaitingScreen.js`

`WaitingScreen`

• Parámetros: **`{ route, navigation }`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async shareCode`

• Parámetros: **`—`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/data/words.js`

`async getRandomWord`

• Parámetros: **`usedWords = []`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`getRandomWordFallback`

• Parámetros: **`usedWords = []`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Módulo: `mobile/hooks/useSocket.js`

`useSocket`

• Parámetros: **`—`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

• No se pudieron inferir entidades del código fuente.

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

• **Lenguaje principal**: JavaScript

• **Archivos detectados**: 21

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
npm install

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
npm start
```

7.3 Despliegue con Docker

```
FROM node:20-slim

WORKDIR /app
COPY . .
RUN npm install
```

```
EXPOSE 8000
CMD ["node index.js"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

• [ ] Variables de entorno configuradas

• [ ] Base de datos migrada

• [ ] Tests ejecutados y pasando

• [ ] Logs configurados

• [ ] HTTPS habilitado en producción

• [ ] Backups programados

• [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-16

• Versión inicial analizada por AutoDocs AI

• Documentación técnica generada automáticamente

• Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 71/100 (Bueno [YELLOW])

Distribución de Documentación:

• Funciones documentadas: 0/42 (0%)

• Clases documentadas: 0/0

Análisis de Complejidad:

• Complejidad promedio: 1.0

• Complejidad máxima: 1

• Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja
```

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

• Agregar docstrings a las 42 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• **Funciones sin documentar:** 42

- `generateRoomCode` en `backend/server.js`

- `getRoomByCode` en `backend/server.js`

- `getUsedWords` en `backend/server.js`

- `maskWord` en `backend/server.js`

- `checkWin` en `backend/server.js`

- ... y 37 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	0	Candidatas a simplificación
■ Baja	≤ 3	42	Complejidad aceptable

■ No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

42 de 42 funciones carecen de docstring (100%).

Función	Archivo	Línea	Complejidad
`generateRoomCode`	`backend/server.js`	?	1
`getRoomByCode`	`backend/server.js`	?	1
`getUsedWords`	`backend/server.js`	?	1
`maskWord`	`backend/server.js`	?	1
`checkWin`	`backend/server.js`	?	1
`serializeRoom`	`backend/server.js`	?	1
`startRound`	`backend/server.js`	?	1
`isValidWord`	`backend/words.js`	?	1
`normalizeWord`	`backend/words.js`	?	1
`fetchFromPalabrasDev`	`backend/words.js`	?	1
`fetchBatchFromPalabrasDev`	`backend/words.js`	?	1
`getRandomWord`	`backend/words.js`	?	1
`testConnection`	`backend/test.js`	?	1
`App`	`mobile/App.js`	?	1
`HangmanDrawing`	`mobile/components/HangmanDrawing.js`	?	1
`Keyboard`	`mobile/components/Keyboard.js`	?	1
`getLetterStatus`	`mobile/components/Keyboard.js`	?	1
`renderKey`	`mobile/components/Keyboard.js`	?	1
`useSounds`	`mobile/app/GameScreen.js`	?	1
`buildInitialSPState`	`mobile/app/GameScreen.js`	?	1

... y 22 funciones más sin documentar.

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **1 grupos de funciones con nombres similares** repartidas en distintos archivos:

Patrón: _getrandomword / getrandomword_ — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad

`getRandomWord`	`backend/words.js`	?	1
`getRandomWord`	`mobile/data/words.js`	?	1

■ **Sugerencia:** Las funciones **`getRandomWord`** y **`getRandomWord`** realizan operaciones similares (**`getrandomword`**) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo **`utils/getrandomword.py`** o **`helpers/common.py`**) que reciba el contexto como parámetro, y llamarla desde ambos archivos (**`mobile/data/words.js`**, **`backend/words.js`**).

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`mobile/app/GameScreen.js`	16	1.0	0%	■ Revisar
`backend/server.js`	7	1.0	0%	■ OK
`backend/words.js`	5	1.0	0%	■ OK
`mobile/components/Keyboard.js`	3	1.0	0%	■ OK
`mobile/app/HomeScreen.js`	3	1.0	0%	■ OK
`mobile/app/WaitingScreen.js`	2	1.0	0%	■ OK
`mobile/data/words.js`	2	1.0	0%	■ OK
`backend/test.js`	1	1.0	0%	■ OK
`mobile/App.js`	1	1.0	0%	■ OK
`mobile/components/HangmanDrawing.js`	1	1.0	0%	■ OK
`mobile/hooks/useSocket.js`	1	1.0	0%	■ OK

■ **`mobile/app/GameScreen.js`** tiene 16 funciones. Considera dividirlo en submódulos más específicos siguiendo el principio de responsabilidad única (SRP).

E) Patrones Frecuentes Adicionales (Análisis IA)

• **get**: 5 ocurrencias

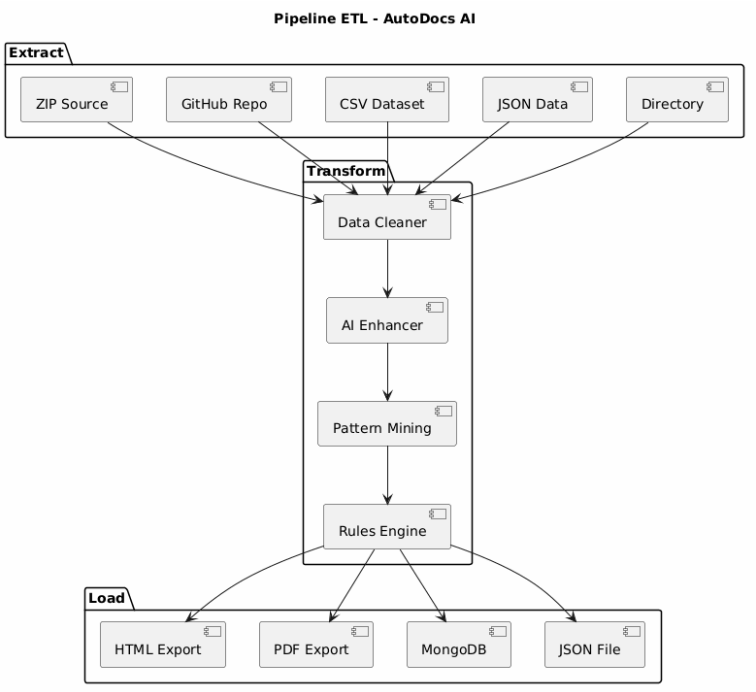
- Ejemplos: ['getRoomByCode', 'getUsedWords', 'getLetterStatus']

• **fetch**: 2 ocurrencias

- Ejemplos: ['fetchFromPalabrasDev', 'fetchBatchFromPalabrasDev']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

• **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales

• **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones

• **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-16 02:39 UTC